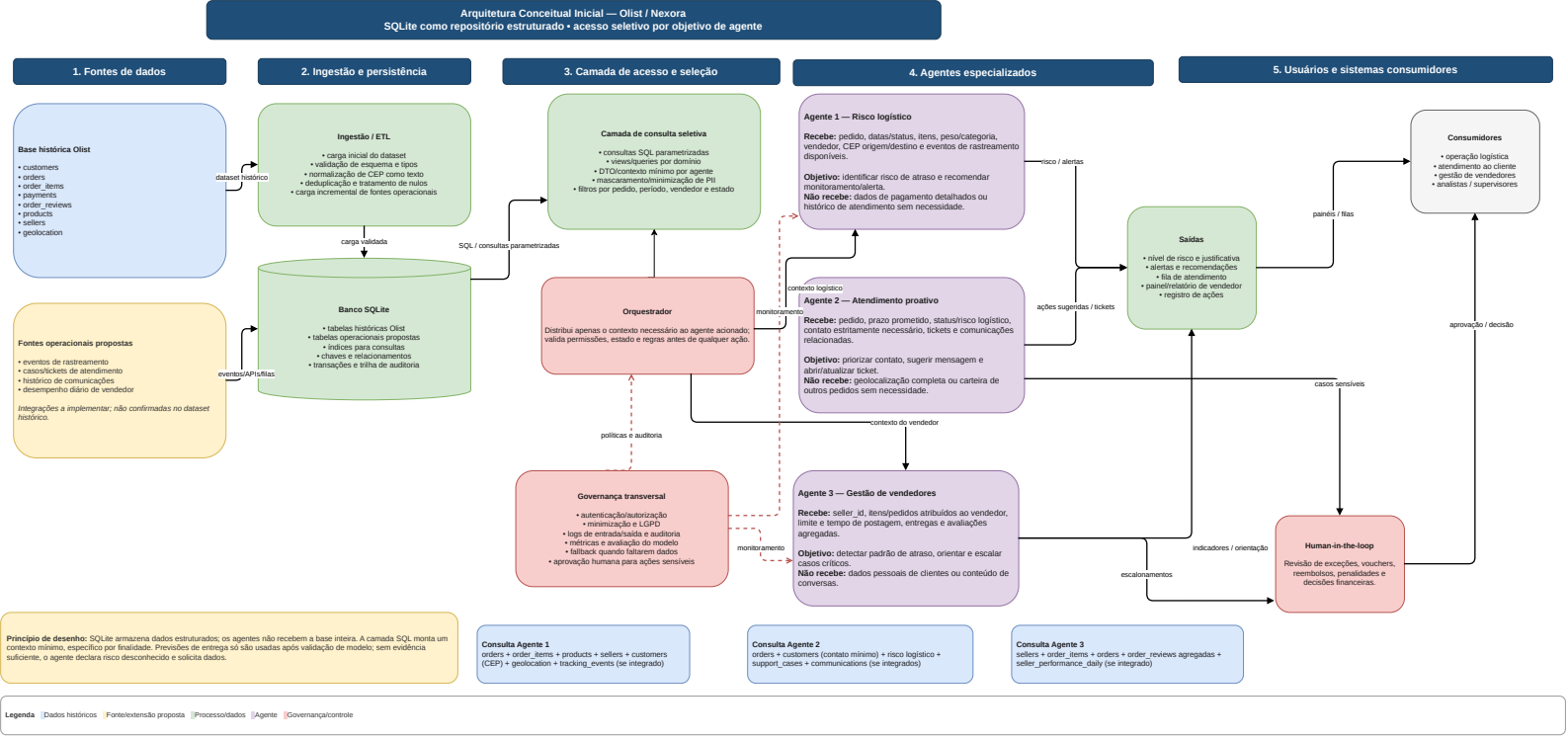

Arquitetura Conceitual Inicial — Olist / Nexora
SQLite como repositório estruturado • acesso seletivo por objetivo de agente
1. Fontes de dados
2. Ingestão e persistência
3. Camada de acesso e seleção
4. Agentes especializados
5. Usuários e sistemas consumidores
Base histórica Olist
• customers
• orders
• order_items
• payments
• order_reviews
• products
• sellers
• geolocation
dataset histórico
Fontes operacionais propostas
• eventos de rastreamento
• casos/tickets de atendimento
• histórico de comunicações
• desempenho diário de vendedor
Integrações a implementar; não confirmadas no dataset histórico.
eventos/APIs/filas
Ingestão / ETL
• carga inicial do dataset
• validação de esquema e tipos
• normalização de CEP como texto
• deduplicação e tratamento de nulos
• carga incremental de fontes operacionais
carga validada
Banco SQLite
• tabelas históricas Olist
• tabelas operacionais propostas
• índices para consultas
• chaves e relacionamentos
• transações e trilha de auditoria
SQL / consultas parametrizadas
Camada de consulta seletiva
• consultas SQL parametrizadas
• views/queries por domínio
• DTO/contexto mínimo por agente
• mascaramento/minimização de PII
• filtros por pedido, período, vendedor e estado
Orquestrador
Distribui apenas o contexto necessário ao agente acionado, valida permissões, estado e regras antes de qualquer ação.
monitoramento
contexto logístico
políticas e auditoria
Governança transversal
• autenticação/autorização
• mínimização e LGPD
• logs de entrada/saída e auditoria
• métricas e avaliação do modelo
• fallback quando faltarem dados
• aprovação humana para ações sensíveis
monitoramento
Agente 1 — Risco logístico
Recebe: pedido, datas/status, itens, peso/categoria, vendedor, CEP origem/destino e eventos de rastreamento disponíveis.
Objetivo: identificar risco de atraso e recomendar monitoramento/alerta.
Não recebe: dados de pagamento detalhados ou histórico de atendimento sem necessidade.
Agente 2 — Atendimento proativo
Recebe: pedido, prazo prometido, status/risco logístico, contato estritamente necessário, tickets e comunicações relacionadas.
Objetivo: priorizar contato, sugerir mensagem e abrir/atualizar ticket.
Não recebe: geolocalização completa ou carteira de outros pedidos sem necessidade.
contexto do vendedor
Agente 3 — Gestão de vendedores
Recebe: seller_id, itens/pedidos atribuídos ao vendedor, limite e tempo de postagem, entregas e avaliações agregadas.
Objetivo: detectar padrão de atraso, orientar e escalar casos críticos.
Não recebe: dados pessoais de clientes ou conteúdo de conversas.
risco / alertas
ações sugeridas / tickets
casos sensíveis
indicadores / orientação
escalonamentos
Saídas
• nível de risco e justificativa
• alertas e recomendações
• fila de atendimento
• painel/relatório de vendedor
• registro de ações
painéis / filas
aprovação / decisão
Consumidores
• operação logística
• atendimento ao cliente
• gestão de vendedores
• analistas / supervisores
Human-in-the-loop
Revisão de exceções, vouchers, reembolsos, penalidades e decisões financeiras.
Princípio de desenho: SQLite armazena dados estruturados; os agentes não recebem a base inteira. A camada SQL monta um contexto mínimo, específico por finalidade. Previsões de entrega só são usadas após validação de modelo; sem evidência suficiente, o agente declara risco desconhecido e solicita dados.
Consulta Agente 1
orders + order_items + products + sellers + customers (CEP) + geolocation + tracking_events (se integrado)
Consulta Agente 2
orders + customers (contato mínimo) + risco logístico + support_cases + communications (se integrados)
Consulta Agente 3
sellers + order_items + orders + order_reviews agregadas + seller_performance_daily (se integrado)
Legenda
Dados históricos
Fonte/extensão proposta
Processo/dados
Agente
Governança/controle